

Contents

Risk Scan — Top 10 — StudioBook (fictional AI-built class-booking app)	1
Scope & method	1
Top 10 findings (of 12 total - the 2 lowest-priority are in the Standard/Advanced tier report, REVIEW_WITH_FIX_PLAN.md)	1
B-01 — Critical	1
B-02 — Critical	2
B-03 — Critical	2
B-04 — Critical	2
B-05 — High	2
B-06 — High	3
B-07 — High	3
B-09 — High	3
B-08 — Medium	3
B-10 — Medium	4
Risk summary (all 12 findings; top 10 by severity detailed above)	4

Risk Scan — Top 10 — StudioBook (fictional AI-built class-booking app)

Deliberately insecure for demonstration and training. Do not deploy. Run locally only.

Prepared: 2026-09-28 · **Tier:** Starter · **Target:** targets/vibe-app (Next.js 14 + Supabase + Stripe, “AI app builder” style baseline)

Scope & method

An automated + manual scan of an AI-generated codebase for the access-control, secrets-handling and business-logic mistakes this style of app commonly ships with. Method: this lab’s M5 scanners (gitleaks, Semgrep custom rules, a Supabase row-level-security checker, Trivy dependency scan) plus manual review and live exploit-script proof (targets/vibe-app/exploits/*.sh) for everything the automated tools structurally can’t see (business-logic races, RLS-adjacent authorization gaps).

Top 10 findings (of 12 total - the 2 lowest-priority are in the Standard/Advanced tier report, REVIEW_WITH_FIX_PLAN.md)

B-01 — Critical

OWASP: A05:2021 - Security Misconfiguration · **API8:**2023 - Security Misconfiguration · **How found:** tool (gitleaks, semgrep)

Evidence:

- .env:20
- components/AdminUserList.tsx:7

The Supabase service-role key (bypasses every RLS policy in the database) is read from a NEXT_PUBLIC_-prefixed environment variable inside a client component. Next.js inlines every NEXT_PUBLIC_* variable into the JavaScript bundle shipped to every visitor’s browser, logged in or not. Anyone who views page source gets a credential that can read or write ANY row in the database, unrestricted.

Fix: Move the query behind a server-only route using the service-role client correctly (never in a “use client” component); delete the NEXT_PUBLIC_ variable entirely.

B-02 — Critical

OWASP: A02:2021 - Cryptographic Failures · API8:2023 - Security Misconfiguration · **How found:** tool (gitleaks)

Evidence:

- `.env:3`
- `lib/stripe.ts:3`

A Stripe secret key is hardcoded directly in a server file, and `.env` (holding the same value plus other credentials) is committed to the repository. Anyone with read access to the repository - a contractor, a leaked backup, an over-shared CI log - obtains a live-shaped credential that can act as the application against Stripe's API.

Fix: Read the secret from an environment variable with no hardcoded fallback (fail to start if unset); add `.env` to `.gitignore` and stop committing it.

B-03 — Critical

OWASP: A01:2021 - Broken Access Control · API1:2023 - Broken Object Level Authorization · **How found:** tool (rls-checker)

Evidence:

- `supabase/migrations/20240101000002_rls_and_storage.sql:39`

Row-level security is never enabled on the bookings table at all. Any authenticated user can read, update, or delete EVERY other user's bookings directly through Supabase's own REST API, completely bypassing the Next.js app's own routes and any checks they perform.

Fix: Enable RLS with an owner/admin SELECT policy; explicitly revoke INSERT/UPDATE/DELETE from `authenticated` - every write must go through a service-role route or RPC, since a policy alone doesn't override Supabase's default table-level grants.

B-04 — Critical

OWASP: A01:2021 - Broken Access Control · API3:2023 - Broken Object Property Level Authorization · **How found:** tool (rls-checker)

Evidence:

- `supabase/migrations/20240101000002_rls_and_storage.sql:10`

RLS IS enabled on profiles, but the SELECT policy is `using (true)` - equivalent to no policy at all for reads. Every authenticated user can read every other user's profile row, including email and phone number - a full customer-PII leak.

Fix: Replace the policy with `id = auth.uid() OR is_admin()`; add a SECURITY DEFINER `is_admin()` helper to avoid the self-referential-policy recursion a plain subquery on profiles would hit.

B-05 — High

OWASP: A01:2021 - Broken Access Control · API1:2023 - Broken Object Level Authorization · **How found:** tool (semgrep)

Evidence:

- `app/api/bookings/[id]/cancel/route.ts:5`

POST `/api/bookings/{id}/cancel` checks that the caller is logged in, but never checks that the booking being cancelled belongs to them. Any authenticated user can cancel any OTHER user's booking by id - a denial-of-service against a specific victim's reservations.

Fix: Filter the update on `id` AND `user_id` (AND `status`, to make a second cancel a no-op); return the same 404 whether the booking belongs to someone else or doesn't exist.

B-06 — High

OWASP: A01:2021 - Broken Access Control · API5:2023 - Broken Function Level Authorization · **How found:** manual

Evidence:

- `app/admin/page.tsx:6`
- `app/api/admin/grant-credits/route.ts:6`

The admin-only 'grant credits' page hides its form from non-admins in the UI, but the route behind it never re-checks the caller's role server-side. Any authenticated user can call the route directly and grant themselves - or anyone - unlimited free credits, bypassing the studio's entire paid-credit model.

Fix: Add a shared `requireAdmin()` helper (reads `profiles.role` via the service-role client, never client-supplied data) and call it before doing anything else in the route, returning 403 before even looking up the target user.

B-07 — High

OWASP: A08:2021 - Software and Data Integrity Failures · API8:2023 - Security Misconfiguration · **How found:** manual

Evidence:

- `app/api/stripe/webhook/route.ts:5`

The Stripe webhook handler parses the request body directly with no signature verification at all. Anyone who knows the webhook URL can POST an arbitrary forged event and have it processed as if Stripe sent it - including a fake `checkout.session.completed` that grants free credits (see B-08).

Fix: Verify the raw body against the stripe-signature header with `stripe.webhooks.constructEvent()` before parsing anything; fail closed if the webhook secret is unset.

B-09 — High

OWASP: A03:2021 - Injection · API8:2023 - Security Misconfiguration · **How found:** manual

Evidence:

- `components/AvatarUpload.tsx:7`
- `supabase/migrations/20240101000002_rls_and_storage.sql:50`

Avatar upload accepts any file type and size to a public storage bucket; an uploaded SVG with an embedded is served back as `image/svg+xml` with no attachment disposition. Stored XSS - a malicious SVG avatar executes its script in the browser of anyone who views it, including via the public bucket URL directly.

Fix: Restrict the bucket's `allowed_mime_types` to a real-image allow-list (no SVG) with a file size cap, and derive the stored filename's extension from the validated type, never the raw uploaded filename.

B-08 — Medium

OWASP: A04:2021 - Insecure Design · API8:2023 - Security Misconfiguration · **How found:** manual

Evidence:

- `app/api/stripe/webhook/route.ts:11`

The webhook handler has no dedup against the Stripe event's own id - POSTing the identical event body twice grants credits twice. A replayed (or maliciously resubmitted) webhook event grants the same credits repeatedly - direct financial loss for the business, scaling with however many times the event is replayed.

Fix: Record processed event ids in a table with a unique constraint, and grant credits in the SAME atomic statement (a SECURITY DEFINER RPC) so there is no window where one succeeded without the other.

B-10 — Medium

OWASP: A04:2021 - Insecure Design · API4:2023 - Unrestricted Resource Consumption · **How found:** manual

Evidence:

- `app/api/bookings/route.ts:20`

The booking route accepts `quantity` with no server-side validation - a negative value is accepted and, combined with the duplicated pricing formula (B-12), actually INCREASES the caller's credit balance instead of charging them. Any authenticated user can generate unlimited free credits for themselves with a single crafted request - direct financial loss.

Fix: Validate quantity is a positive integer within a sane bound before it reaches the database; move the booking + credit debit into one atomic, capacity/balance-checking database function.

Risk summary (all 12 findings; top 10 by severity detailed above)

Severity	Count
Critical	4
High	4
Medium	3
Low	1
Total	12

Full findings, evidence, remediation plan and retest notes: `REVIEW_WITH_FIX_PLAN.md` (Standard/Advanced tier).